Q1. Write programs according to given description: **(10+35+15+10+10 = 80 Points)**

a. Write copy constructor, assignment operator and destructor. To secure full marks avoid duplication of code.

```
class Details{
   int d1, d2, d3;//d1, d2, d3 are working days in three months
   int *s1, *s2, *s3;//to store sales of each day in three months
```

b. Please write a complete class named **Meal** with the required members. A meal consists of three options: chicken (1), vegetable (2), or mix (3). Additionally, the meal is available in three sizes: small (s), medium (m), or large (l). Salad may or may not be part of the meal. There are three options for drinks: plain water (p), juice (j), or soft drink (s). Each meal order should have an order number that begins at one and increases sequentially for each next order. The default values for a meal are chicken, medium size, with salad and plain water. Define constants to ensure enhanced readability of your code and receive full marks. Additionally, include the following member functions:

- Parameterized constructor with default values for all parameters. Call setter function to validate and set only legal values
- individual setter functions for all data members, with default value of parameters and ensure to check for legal values
- getter functions for size and drink only
- overload stream insertion operator to show output like:
- overload stream extraction operator to read the object members from both the keyboard and the file, assuming each data members is on a different line.
- Write main function to create two objects with different parameters. Ensure to call all member functions at least once. Finally, provide the write output of your code.

```
Order:   5
Type:    Chicken
Size:    Medium
Salad:   Yes
Drink:   Juice
```

c. For class MyList, write following member functions:

```
class MyList{ //C is used for const
   int size, cSize;
   int *d;
 public:
   MyList(int s = 10)...
   MyList(C MyList &m) ...
   MyList& operaor+=(C MyList &m)...
   //all getter setters
   ...
```

- Overload stream operator without friendship to print object like:

```
Size : 10   Current Size : 5
23 45 19 8 -37
```

- Overload operator **>**, return **true** if **sum** of the elements of first object is greater than the **sum** of the elements of second object, otherwise return false
- Overload operator **--** (post decrement) to subtract one from all the elements
- **`MyList removeDup ()`** to return a new object without the duplication of elements

d. For class MyString, write following member functions:

- **`MyString removeInts ()`** to return a new string object without the integers
- **`MyString intSum (MyString &)`** to return a new object having sum of first two objects. For this function assume that both strings have only digits from 0 to 9, starting from a non-zero digit.

```
class MyString{ //C is used for const
   int size;
   char *c;
 public:
   //all constructors
   //all getter setters
   ...
```

e. Write code to read file "**scores.txt**" using dynamic memory allocation and deallocation read file and print average (only) of each player.

```
scores.txt
n  (Number of players)
m1  (count of matches for player 1)
43  28  …  (scores of player 1 with spaces)
m2  (count of matches for player 2)
19  …  (scores of player 2 with spaces)
...
```

Q2. Write output of the following code: **(25 Points)**

```
class OddEven{
      int odd, even;
   public:
      OddEven(int o = 1, int e = 2){
         cout << "C ";
         odd = o;
         even = e;
      }
      OddEven(const OddEven &oe){
         cout << "P ";
         odd = oe.odd + 2;
         even = oe.even + 4;
      }
      OddEven& operator = (const OddEven &oe){
         cout << "H ";
         odd = oe.odd * 2;
         even = oe.even;      return *this;
      }
      ~OddEven()     {  cout << "D " << *this;  }
      friend ostream& operator << (ostream &out, const OddEven &oe){
         out << oe.odd << '-' << oe.even << '\n';
         return out;
      }
      OddEven& operator += (int x){
         cout << "B ";
         odd = odd + 4; return *this;
      }
      OddEven& operator += (const OddEven &oe){
         cout << "A ";
         even = oe.even + 1;  return *this;
      }
      void ff() const{
         cout << "G ";
      }
};
void funfun(OddEven oe){
   OddEven o1;
   OddEven *o2 = new OddEven(o1);
   cout << *o2;
}
void operator *(OddEven &o1, OddEven &o2){
   cout << "K ";
   o1 += 2;
}
int main(){
   OddEven oe[3];
   oe[0]+=oe[2];
   oe[2] = oe[1];
   cout << oe[0] << oe[1] << oe[2];
   funfun(oe[1]);
   OddEven *oo = new OddEven(1,2);
   cout << *oo;
   oo->ff();
   delete oo;
   oe[0] * oe[1];    return 0;}
```

Q3. Please find and identify any logical or syntax errors. For each error discovered, clearly indicate SE for Syntax Error and LE for Logical Error. Additionally, describe the nature of error. If you only underline and write SE & LE without further explanation, you will receive zero marks: (15 Points)

```
class Error{
      int x;
   public:
      Error(){
         x = 1;
      }
      Error(int x = 1){
         x = this->x;
      }
      Error(const Error &e){
         x = e.x;
      }
      Error& operator = (const Error &e){
         x = e.x;    return *this;
      }
      int getX(){
         return x;
      }
      friend ostream& operator << (ostream &out, Error &e){
         cout << e.x << '\n';
         return out;
      }
      Error operator + (Error &e){
         Error er;
         er.x = x + e.x;
         return er;
      }
      Error& operator += (int x){
         this->x += x;
         return *this;
      }
      Error& operator += (const Error &e){
         x += e.x;            return *this;
      }
      static void about() const{
         cout << "Error Class\n";
      }
};
void globalFun(Error &e){     cout << x ;   }
void operator *=(Error &o1, Error &o2){
   o1.x *= o2.x;
}
int main(){
   Error oe[3];
   cout << oe[0] + oe[2];
   oe[2] = oe[1];
   cout << oe[0] << oe[1] << oe[2];
   globalFun(oe[1]);
   Error *oo = new Error(5);
   cout << *oo;
   oo->about();
   oe[0] *= oe[1];      return 0;}
```